MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory Works No. 9 and No. 10

Watching System Calls with `strace` (Linux)
Adding the `trace` System Call (xv6)

Student: Tacu Igor, FAF-242
Checked by: Reițman Patricia

UTM · Chișinău · 2026

Purpose and environment

Laboratory Work No. 9 observes Linux system calls from outside a process with `strace`. Laboratory Work No. 10 adds a `trace(int)` system call to xv6, so the operating system itself can report calls selected by a bit mask. The Linux work was run as `root` on Ubuntu in `~/os-lab9`. The xv6 work was built in `~/xv6-riscv` and run in QEMU.

Laboratory Work No. 9 — Linux

Tracing a real command

The command `strace -f -e trace=openat,read,write,close ls -l` showed that even a simple directory listing opens libraries, locale files, and the current directory. Output sent to the terminal uses `write` on descriptor 1 (standard output). The separate count `strace -c -e trace=openat ls -l` reported **31 `openat` calls**, including **18 failed path lookups**. The failed lookups were part of normal environment setup; `ls` still completed.

```
root@ubuntu-4gb-hel1-1:~# mkdir -p ~/os-lab9
root@ubuntu-4gb-hel1-1:~# cd ~/os-lab9
root@ubuntu-4gb-hel1-1:~/os-lab9# command -v strace || apt install strace
/usr/bin/strace
root@ubuntu-4gb-hel1-1:~/os-lab9# strace -f -e trace=openat,read,write,close ls -l 2>&1 | tee trace-ls.txt
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0"..., 832) = 832
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libpcre2-8.so.0", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3)                                = 0
openat(AT_FDCWD, "/proc/filesystems", O_RDONLY|O_CLOEXEC) = 3
read(3, "nodev\tsysfs\nnodev\ttmpfs\nnodev\tbd"..., 1024) = 414
read(3, "", 1024)                       = 0
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/share/locale/locale.alias", O_RDONLY|O_CLOEXEC) = 3
read(3, "# Locale name alias data base.\n#"..., 4096) = 2996
read(3, "", 4096)                       = 0
close(3)                                = 0
openat(AT_FDCWD, "/usr/share/locale/en_US.UTF-8/LC_TIME/coreutils.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/share/locale/en_US.utf8/LC_TIME/coreutils.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/share/locale/en_US/LC_TIME/coreutils.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/share/locale/en.UTF-8/LC_TIME/coreutils.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/share/locale/en.utf8/LC_TIME/coreutils.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/share/locale/en/LC_TIME/coreutils.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
close(4)                                = 0
close(4)                                = 0
openat(AT_FDCWD, "/etc/nsswitch.conf", O_RDONLY|O_CLOEXEC) = 4
read(4, "# /etc/nsswitch.conf\n#\n# Example"..., 4096) = 526
read(4, "", 4096)                       = 0
close(4)                                = 0
openat(AT_FDCWD, "/etc/passwd", O_RDONLY|O_CLOEXEC) = 4
read(4, "root:x:0:0:root:/root:/bin/bash\n"..., 4096) = 1798
close(4)                                = 0
close(4)                                = 0
close(4)                                = 0
openat(AT_FDCWD, "/etc/group", O_RDONLY|O_CLOEXEC) = 4
read(4, "root:x:0:\ndaemon:x:1:\nbin:x:2:\ns"..., 4096) = 806
close(4)                                = 0
```

Figure 1: Selected Linux calls made by `ls -l`.

Calling the kernel from a C program

The program `syscalls.c` used `open`, `write`, `read`, and `close` to create `note.txt`, write `hello, kernel` and read it back. It reported descriptor **3**, **14 bytes written**, and **14 bytes read**. Descriptors 0, 1, and 2 are normally reserved for standard input, output, and error, so the first new file received descriptor 3. `strace` showed the C `open` calls as Linux `openat` calls. The program exited with status 0.

```
root@ubuntu-4gb-hel1-1:~/os-lab9# gcc -std=c11 -Wall -Wextra -o syscalls syscalls.c
root@ubuntu-4gb-hel1-1:~/os-lab9# ./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
root@ubuntu-4gb-hel1-1:~/os-lab9# strace -e trace=openat,write,read,close ./syscalls 2>&1 | tee trace-syscalls.txt
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0"..., 832) = 832
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(3, "hello, kernel\n", 14)         = 14
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_RDONLY)  = 3
read(3, "hello, kernel\n", 63)          = 14
close(3)                                = 0
write(1, "open returned descriptor 3\nwrite"..., 80open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
) = 80
+++ exited with 0 +++
root@ubuntu-4gb-hel1-1:~/os-lab9# strace -c -e trace=openat ls -l >/dev/null
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
100.00    0.000401          12        31        18 openat
------ ----------- ----------- --------- --------- ----------------
100.00    0.000401          12        31        18 total
root@ubuntu-4gb-hel1-1:~/os-lab9#
```

Figure 2: The program output, its file calls, and the `openat` count.

Laboratory Work No. 10 — xv6

Adding the system call

`SYS_trace` was assigned number **23**. The user declaration, assembly stub, and `trace` utility issue the call with a mask, then replace the utility with the requested command using `exec`. The kernel stores the mask in `struct proc`, copies it during `fork`, dispatches `sys_trace`, and prints the selected call name and return value after the call completes.

Before the kernel handler was added, `trace 32 grep hello README` produced `unknown sys call 23`. This confirmed that the user stub reached the kernel. After the handler was added, the same command printed four `read` results: **1023**, **965**, **453**, and **0**. The first three total **2441 bytes**; the final zero denotes end of file.

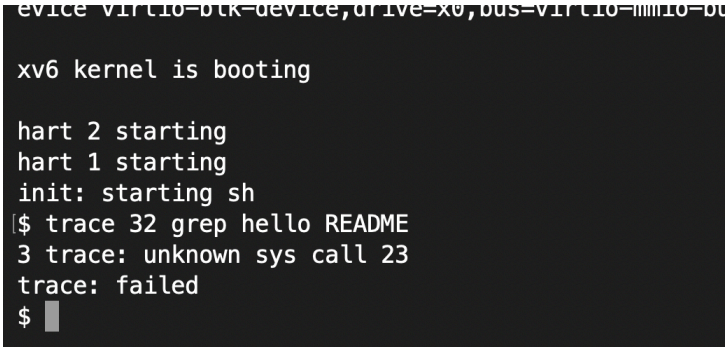


Figure 3: The intermediate check before registering the kernel handler.

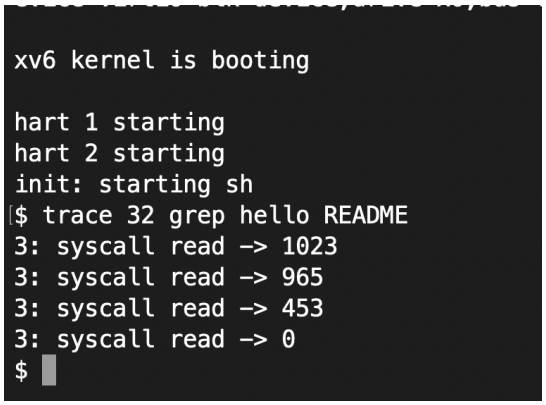


Figure 4: Successful tracing of `read` with mask 32.

Testing masks and inheritance

Each bit in the mask selects a system call number. Mask **2147483647** displayed all calls used by `grep`, including `trace`, `exec`, `open`, `read`, and `close`. Mask **65536** selected `write` (call 16): `echo hi` performed one write for `hi` and another for the newline, so a trace line appeared between them. Mask **2129920** selected only `open` (15) and `close` (21): $(1 \ll 15) \mid (1 \ll 21)$.

The `trace 2 usertests forkforkfork` test selected only `fork` (call 1). Lines from several PIDs confirmed that children inherited the mask through `np->trace_mask = p->trace_mask`. The test reached `fork -> -1` when the process table filled and ended with **OK** and **ALL TESTS PASSED**.

```
$ trace 2147483647 grep hello README
4: syscall trace -> 0
4: syscall exec -> 3
4: syscall open -> 3
4: syscall read -> 1023
4: syscall read -> 965
4: syscall read -> 453
4: syscall read -> 0
4: syscall close -> 0
$
$ trace 65536 echo hi
hi5: syscall write -> 2

5: syscall write -> 1
$
$ trace 2129920 grep hello README
6: syscall open -> 3
6: syscall close -> 0
$
$ trace 2 usertests forkforkfork
usertests starting
test forkforkfork: 7: syscall fork -> 8
8: syscall fork -> 9
9: syscall fork -> 10
9: syscall fork -> 11
9: syscall fork -> 12
10: syscall fork -> 13
9: syscall fork -> 14
10: syscall fork -> 15
9: syscall fork -> 16
9: syscall fork -> 17
9: syscall fork -> 18
9: syscall fork -> 19
10: syscall fork -> 20
9: syscall fork -> 21
10: syscall fork -> 22
10: syscall fork -> 23
10: syscall fork -> 24
```

Figure 5: All-call, write-only, open-and-close, and fork-only masks.

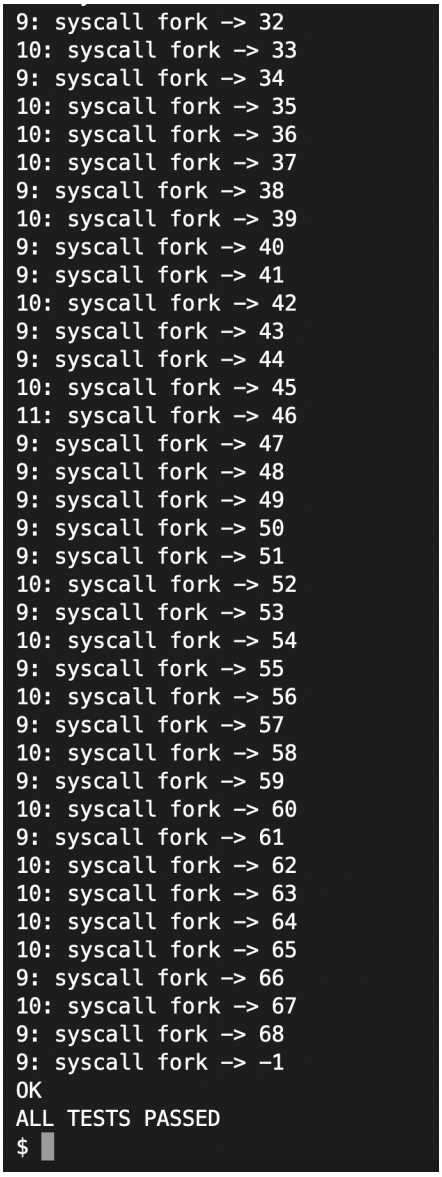


Figure 6: The fork test reaches the process limit and passes.

Conclusion

The Linux experiment showed that C library calls resolve to kernel operations on numbered descriptors. The xv6 experiment implemented the same observation mechanism inside the kernel. The mask controlled which calls were printed, `exec` preserved the selected mask, and `fork` passed it to children. The Lab 10 program compiled, booted, and passed its functional tests.

The xv6 implementation is published on the personal fork's `lab10-trace` branch; the observed Linux program and answers are retained alongside this report.